

QA 4 (Unit 4)

Clustering Implementation and Visualization

K-Means and DBSCAN on Customer Data

Subject: Machine Learning

1. Introduction

Clustering is an unsupervised machine learning technique used to group similar data points together without predefined labels. This assignment implements and compares two prominent clustering algorithms:

- K-Means Clustering — a centroid-based partitioning algorithm
- DBSCAN — a density-based algorithm that can detect noise and arbitrary-shaped clusters

The dataset used is a synthetic customer dataset with two features: Annual Income (k\$) and Spending Score (1–100). A total of 305 data points were generated, including 5 intentional outliers.

2. Dataset Description

2.1 Features

- Annual Income (k\$): Ranges roughly from 5 to 100
- Spending Score (1–100): Reflects customer spending behavior

2.2 Data Generation

The dataset was generated using `sklearn.datasets.make_blobs` with 3 natural cluster centers:

- [30, 20] — Low income, low spending (budget-conscious segment)
- [60, 60] — Mid income, mid spending (mainstream segment)
- [85, 85] — High income, high spending (premium segment)

Five additional outlier points were manually added to test each algorithm’s ability to handle noise.

3. Preprocessing Steps

3.1 Standardization

Both K-Means and DBSCAN rely on Euclidean distance. Without scaling, features with larger ranges dominate the distance calculation. We applied `StandardScaler` to transform both features to zero mean and unit variance.

Metric	Before Scaling	After Scaling
Mean (Income)	~58.5	≈0.00

Std (Income)	~22.0	~1.00
Mean (Spending)	~55.0	≈0.00
Std (Spending)	~28.5	~1.00

4. K-Means Clustering

4.1 Algorithm Overview

K-Means works as follows:

1. Initialize K centroids randomly
2. Assign each point to the nearest centroid (Euclidean distance)
3. Recompute centroids as the mean of all assigned points
4. Repeat steps 2–3 until convergence (centroids no longer move)

4.2 Choosing Optimal K

We used two methods to determine the best number of clusters:

- Elbow Method: Plot inertia (within-cluster sum of squares) vs K. The "elbow" at K=3 indicates diminishing returns beyond 3 clusters.
- Silhouette Score: Measures how similar a point is to its own cluster vs neighbouring clusters. Higher is better. Score peaked at K=3.

4.3 Results

Cluster	Size	Interpretation
Cluster 0	~100 pts	Low income, low spending
Cluster 1	~100 pts	Mid income, mid spending
Cluster 2	~105 pts	High income, high spending

Silhouette Score (K-Means): ~0.61 — indicating well-separated, compact clusters.

Limitation: All 5 outlier points were forcefully assigned to the nearest cluster, slightly distorting centroids.

5. DBSCAN Clustering

5.1 Algorithm Overview

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) classifies points into three categories:

- Core Points: Have at least min_samples neighbours within radius eps
- Border Points: Within eps of a core point but not core themselves
- Noise Points: Neither core nor border (labelled -1)

Key parameters used: eps = 0.5, min_samples = 5. The eps value was selected using the k-distance graph, where the "knee" point indicates a suitable neighbourhood size.

5.2 Results

Label	Points	Interpretation
Cluster 0	~100 pts	Low income, low spending
Cluster 1	~100 pts	Mid income, mid spending
Cluster 2	~100 pts	High income, high spending
Noise (-1)	5 pts	Outliers / anomalies

DBSCAN Silhouette Score (non-noise): ~0.62 — comparable to K-Means, with the added benefit of noise detection.

6. K-Means vs DBSCAN — Comparison

Feature	K-Means	DBSCAN
Cluster Shape	Spherical / convex	Arbitrary shape
K required?	Yes (must be specified)	No (discovered automatically)
Noise handling	None (all points assigned)	Labels outliers as -1
Key parameters	n_clusters	eps, min_samples
Sensitivity	Sensitive to initialisation	Sensitive to eps value
Scalability	Good ($O(nKt)$)	Moderate ($O(n \log n)$ with tree)
Silhouette Score	~0.61	~0.62 (non-noise pts)

7. Analysis and Observations

7.1 Which Algorithm Performed Better?

For this dataset, both algorithms successfully identified the 3 natural customer segments. However, DBSCAN provides a more complete picture for the following reasons:

- It correctly identified and isolated the 5 outlier/noise points rather than forcing them into a cluster, which is critical for real-world business intelligence.
- DBSCAN requires no prior assumption about the number of clusters; this is useful when K is unknown.
- The silhouette scores are nearly identical, confirming both algorithms found equally compact, well-separated clusters.

K-Means is the preferred choice when: the data is clean (few outliers), clusters are roughly spherical, and execution speed matters at scale.

7.2 Business Insights from Clusters

- Cluster 0 — Budget Segment: Low-income, low-spending customers. Marketing strategy: focus on value deals and discounts.
- Cluster 1 — Mainstream Segment: Mid-tier customers. Strategy: loyalty programmes and moderate upselling.
- Cluster 2 — Premium Segment: High-income, high-spending customers. Strategy: exclusive offers, premium products, VIP treatment.
- Noise Points: Anomalous customers who do not fit any segment. Worth investigating for data quality or fraud detection.

7.3 Limitations

- K-Means assumes spherical clusters and equal cluster sizes, which may not reflect real customer data.
- DBSCAN's performance is highly sensitive to the `eps` and `min_samples` hyperparameters.
- Both algorithms use Euclidean distance, which may not be ideal for all feature types.

8. Conclusion

This assignment demonstrated the implementation and comparison of K-Means and DBSCAN clustering on a synthetic customer dataset. Key takeaways:

- K-Means cleanly partitioned customers into 3 well-defined segments using centroid-based assignment.
- DBSCAN discovered the same 3 clusters while additionally flagging 5 noise points, offering a more nuanced view of the data.

- Both algorithms achieved comparable Silhouette Scores (~ 0.61 – 0.62), validating the quality of the cluster separation.
- For noisy, real-world data with unknown cluster counts, DBSCAN is the more robust choice. For clean, structured data at scale, K-Means is preferred.

9. References

- Scikit-learn Documentation — <https://scikit-learn.org/stable/modules/clustering.html>
- Ester, M. et al. (1996). A density-based algorithm for discovering clusters. KDD-96 Proceedings.
- MacQueen, J. (1967). Some Methods for Classification and Analysis of Multivariate Observations. Berkeley Symposium.
- Rousseeuw, P.J. (1987). Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis.